



ISA - TFTP dokumentace

Jiří, Prokop
xproko47

18. listopadu 2023

Obsah

1	Uvedení do problematiky	3
1.1	Co to je TFTP	3
1.2	WRQ	3
1.3	RRQ	3
1.4	Ukončení	3
1.5	Mode	4
1.6	Rozšíření options	4
1.7	option blocksize	4
1.8	options transfer size a timeout interval	4
2	Návrh aplikace	5
3	Popis implementace	5
3.1	soubor tftp_server	5
3.2	soubor tftp_client	5
3.3	soubor tftp	5
4	Základní informace o programu	6
5	Návod na použití	6
5.1	tftp-server	6
5.2	tftp-client	7
6	Zdroje	8

Uvedení do problematiky

Co to je TFTP

TFTP nebo také Trivial File Transfer Protocol je velice jednoduchý protokol pro přenos souborů, obsahující jen základní funkce protokolu FTP. TFTP je určen pro přenos souborů v případech, kdy je běžný protokol FTP nevhodný pro svou komplikovanost. Typickým případem je bootování bezdiskových počítačů ze sítě (viz také BOOTP), kdy se celý přenosový protokol musí vejít do omezeného množství paměti, která je k dispozici na bezdiskovém stroji. 1

V rámci zadání jsme měli implementovat server i klienta podle RFC 1350(viz 2). Tento protokol určuje postup komunikace mezi serverem a klientem a za tímto účelem definuje 5 typů packetů:

- RRQ
- WRQ
- DATA
- ACK
- ERROR

Konkrétní tvar těchto packetů nebude součástí tohoto textu, ale případný zájemce je může najít v uvedeném RFC. RRQ a WRQ slouží pro započetí komunikace klienta se serverem. WRQ je žádost o zápis souboru od klienta na server a RRQ o čtení souboru(ze serveru ke klientovi). DATA packet slouží k poslání dat souboru. ACK slouží ve většině případů k potvrzení data packetů(upřesněno později) a error je určen pro zaslání informace o erroru.

Aplikace je navržena pro implementaci pomocí protokolu UDP, takže musí obsahovat vlastní 'řízení' spojení. Z tohoto důvodu se vždy - jak na serveru tak na klientovi, každý krok musí potvrdit druhou stranou. Podle dřívější verze standardu protokolu, mohla strana reagovat i na stará potvrzení¹ i přesto, že už přišla novější. To mohlo způsobit tzv. 'Sorcerer's Apprentice Syndrome'(více viz 3). Z tohoto důvodu už strana reaguje pouze na nejnovější potvrzení.

Celý flow funguje následovně. Client zašle serveru request packet na stanovený port. Server odpoví z nového portu, určenému pouze pro tohoto klienta. Komunikaci lze následně rozdělit do dvou typů, podle klientova requestu.

WRQ

Pokud šlo o write request, server odpoví ACK packetem s block číslem 0. Klient následně začne posílat data. Jak již bylo dříve zmíněno, každý data packet musí být potvrzen ACK packetem s příslušným block číslem.

RRQ

V případě read requestu odpoví server prvním DATA packetem. Ten musí být od klienta potvrzen ACK packetem.

Ukončení

V případě žádné chyby se konec komunikace pozná tak, že DATA packet je menší než je jeho dohodnutá velikost. Nastane-li chyba, pošle strana straně druhé ERROR packet. Tento packet se nepotvrzuje, takže se strana po jeho odeslání může opustit komunikaci. Jelikož se ale jedná o komunikaci přes UDP

¹stará ve významu nižšího block čísla, než jaké bylo už od druhé strany potvrzeno

není jisté, že packet dorazí. Proto musí mít obě strany aktivní timeout. Jakmile vyprší mohou opustit komunikaci nebo se pokusit o znovuodeslání posledního packetu. Timeout musí při každém takovém pokusu exponenciálně navýšen, jak je uvedeno v RFC 1123(viz 4).

Mode

TFTP může operovat ve třech módech:

- octet
- netascii
- mail

Mail mode je v RFC 1350(viz 2) označen za 'obsolete' a neměl by být implementován. V octet módu se posílají data, tak jak je druhá strana pošle - beze změny. Při netascii módu je třeba před odesláním převést data z přirozeného formátu operačního systému, na kterém je aplikace spouštěna. Pro aplikace běžící na linuxu to znamená převést všechny znaky '`\r`' na '`\r\0`' a '`\n`' na '`\r\n`'.

Mód je určen v request packetu.

Rozšíření options

Podle zadání projektu, bylo rovněž třeba implemenovat tři rozšíření. Jedná se o následující:

- blocksize
- timeout interval
- transfer size

Dohodnutí rozšíření se řídí podle standardu 2347(viz 5). Jednotlivé options navrhuje pouze klient, server je může pouze schválit či zamítnout. Návrhy na rozšíření jsou přiložena k WRQ nebo RRQ. Pokud server zavrhne všechna navržená rozšíření, chová se jako bez nich. Pro jejich potvrzení byl zaveden nový typ packetu OACK, který je detailněji popsán v uvedeném RFC. Pokud klient začal komunikaci WRQ a server souhlasí alespoň se některým rozšířením, zašle klientovi OACK, ve kterém uvedeno odsouhlasené options. Klient vybrané options potvrdí DATA packetem a v opačném případě zašle ERROR packet a opouští komunikaci. V případě RRQ server postupuje stejně, pouze klient provede finální potvrzení options pomocí packetu ACK s block číslem 0.

option blocksize

Podle RFC 2348(viz 6). Block size option je určena pro stanovení velikosti DATA packetu. Standardní TFTP totiž používá konstantní velikost 512 oktetů. Díky tomuto rozšíření se jeho velikost může změnit na 8 až 65464 oktetů. Server vždy může navrhnout hodnotu nižší nebo stejnou, jako nabídnul klient.

options transfer size a timeout interval

Podle RFC 2349(viz 7). V rámci transfer size option se zašle celková velikost dat před tím, než jsou poslána, takže druhá strana může reagovat errorem z důvodu nedostatečné paměti.

Díky timeout rozšíření se může server s klientem dohodnout na stejné velikosti timeoutu.

Návrh aplikace

Na začátku je třeba se rozhodnout zda budeme tvořit iterativní nebo konkurenční server. Po rozvaze jsem zvolil server konkurenční. Pro implementaci TFTP protokolu je totiž vhodnější hned z několika důvodů. Předně, protokol stanovuje, že hlavní server běží na zvoleném portu (defaultně 69) a pro každou komunikaci si zvolí jiný náhodný port. Konkurenční přístup proto nabízí snadnou implementaci tohoto procesu, kde hlavní proces běží na zvoleném portu a při obdržení request packetu vytvoří nový proces, který si zabere nový port a obslouží zbytek komunikace s klientem. Po erroru nebo úspěšném ukončení se proces ukončí a hlavní proces ho uzavře. Jelikož RFC standard TFTP (viz 2) nepovoluje přepis existujících souborů, koordinaci procesů není třeba příliš řešit. Další výhodou je snadnější udržení operativnosti serveru. Pokud se totiž při komunikaci s klientem objeví vážná chyba - např. SEGMENTATION FAULT, havaruje pouze proces obsluhující tuto komunikaci, ale hlavní proces serveru zůstane netknutý a je nadále operativní. Konkurenční server také snadněji obsloužit více klientů zároveň.

Projekt jsem dále rozdělil do 3 logických částí: klient, server a sdílené funkce. Tomu odpovídá i rozdělení do jednotlivých souborů. Ve sdílené části se nachází hlavně pomocné funkce vytvářející TFTP packety a funkce pro čtení souboru a převádění do nebo z netascii. Z tohoto důvodu jsem soubor obsahující sdílený kód pojmenoval jednoduše `tftp`.

Popis implementace

K popisu jednotlivých funkcí slouží primárně komentáře. Zde zmíním pouze zajímavější pasáže a hlavní flow.

soubor `tftp_server`

Poté co server dokončí setup přejde do hlavní smyčky. Hlavní smyčka serveru funguje následovně. Na začátku čeká na zvoleném portu na jakýkoliv packet. Po tom co nějaký obdrží, zavolá server funkci `parse_req_packet`, která naparsuje packet a zkontroluje jeho strukturu. Po tom, co se zkontrolují naparované argumenty z request packetu, se vytvoří nový proces a podle typu requestu zavolá funkce `tftp_read` nebo `tftp_write`, kde pokračuje komunikace s klientem. Hlavní proces mezitím zkontroluje, jestli nejsou k ukončení nějaké zombie procesy a vrací se na začátek smyčky.

soubor `tftp_client`

Klient funguje v principu obdobně jako server. Hlavní rozdíl je, že klientovi stačí pouze jeden proces. Také musí navíc na začátku zaslat serveru request packet. Dalším rozdílem je, že klient nepodporuje žádná rozšíření. Z tohoto důvodu bylo zapotřebí testovat server, pomocí externího klienta, který rozšíření podporuje. Použil jsem `atftp` (viz 8).

soubor `tftp`

Jak už jsem zmínil, v tomto souboru se nachází všechny sdílené pomocné funkce. Za zmínku stojí funkce `get_nchars_from_file`, která slouží k načtení dat do připraveného bufferu. V principu by nešlo o nic složitějšího, pokud by neexistoval netascii mode. Funkce postupně čte a zapisuje každý znak a na závěr vrátí počet načtených znaků, což je potřeba pro detekci konce souboru a tedy pro ukončení komunikace s druhou stranou. Pokud se ale při čtení narazí na znak `'\r'` nebo `'\n'` musí zapsat o znak navíc - v případě CR `'\r\0'` a LF `'\r\n'`. Může se ale stát, že na zapsání druhého znaku už není v bufferu místo. Z toho důvodu je proměnná uchovávající znak *static*, takže si zachová svoji hodnotu do dalšího zavolání funkce. Její hodnota se poté zkontroluje, aby se nahrál druhý znak.

Funkce `get_nchars_from_file`:

```
size_t get_nchars_from_file(FILE *fp, size_t n, char *buf, int mode) {
    static int c = 0;
    size_t i = 0;
    // put left over letter from last reading
    if (mode == netascii_mode) {
        if (c == '\n') {
            buf[i++] = c;
        } else if (c == '\r') {
            buf[i++] = '\0';
        }
    }
    for (; i < n; i++) {
        c = fgetc(fp);
        if (mode == netascii_mode && (c == '\r' || c == '\n')) {
            buf[i] = '\r';
            if (++i == n) {
                return i;
            }
            if (c == '\r') {
                c = '\0';
            }
        }
        if (c == EOF) {
            return i;
        }
        buf[i] = c;
    }
    return i;
}
```

Základní informace o programu

Tento projekt zahrnuje vývoj sady nástrojů pro TFTP (Trivial File Transfer Protocol), která se skládá z TFTP serveru (`tftp-server`) a TFTP klienta (`tftp-client`). TFTP server je navržen k přijímání a ukládání souborů od vzdálených klientů, zatímco TFTP klient usnadňuje stahování a nahrávání souborů na vzdálený server.

Pro přeložení programu stačí napsat `make`. Pro odstranění spustitelných souborů `make clean`.

Návod na použití

`tftp-server`

`tftp-server [-p port] root_dirpath`

`-h, --help:` Vytiskne nápovědu.

`-p:` Určuje lokální port, na kterém má server běžet.
Výchozí hodnota je 69.

`root_dirpath:` Cesta k adresáři, kam budou ukládány příchozí soubory.

Příklad: `./tftp-server -p 7999 .`

Pokud si zvolíte výchozí port, je nutné program spustit s právy správce (sudo), protože port 69 je součástí známých portů.

tftp-client

`tftp-client -h hostname [-p port] [-f filepath] -t dest_filepath`

`--help` Vytiskne nápovědu.

`-h:` IP adresa/doménové jméno vzdáleného serveru.

`-p:` Port vzdáleného serveru. Výchozí hodnota je 69.

`-f:` Cesta k souboru, který má být stažen ze serveru (download).
Pokud není specifikováno, obsah bude načítán ze vstupu (upload).

`-t:` Cesta, pod kterou bude soubor uložen na vzdáleném serveru
nebo lokálně.

Příklad (upload): `./tftp-client -h localhost -p 7999 -t some_name.txt <myfile.txt`

Zdroje

1. https://cs.wikipedia.org/wiki/Trivial_File_Transfer_Protocol
2. <https://datatracker.ietf.org/doc/html/rfc1350>
3. https://en.wikipedia.org/wiki/Sorcerer%27s_Apprentice_Syndrome
4. <https://datatracker.ietf.org/doc/html/rfc1123#page-46>
5. <https://datatracker.ietf.org/doc/html/rfc2347>
6. <https://datatracker.ietf.org/doc/html/rfc2348>
7. <https://datatracker.ietf.org/doc/html/rfc2349>
8. <https://linux.die.net/man/1/atftp>